S.No: 1	Exp. Name: ***Eldest of Three***	**Date: 2026-08-04**

Aim:

In a classroom, a teacher wants to identify the eldest student among the three. To simplify this task, the teacher decides to write a Python program that takes the ages of the three students and determines the oldest.

Can you help the teacher by writing this program?

Input Format:

- Three integers are entered on separate lines, each representing the age of a student.

Output Format:

- The output is the integer that represents the age of the elder student.

Source Code:

eldestAge.py

```
# Write your code here..
a=int(input())
b=int(input())
c=int(input())
if(a>b and a>c):
        print(a)
elif(b>a and b>c):
        print(b)
else:
        print(c)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
12
14
13
14

Test Case - 2
User Output
11
43
23
43

S.No: 2	Exp. Name: ***Prime numbers from 2 to the given positive number***	**Date: 2026-08-04**

Aim:

Implement a program that takes an integer n input from the user and prints all the prime numbers from 2 to n.

Note: A prime number is divisible only by 1 and itself.

Input format:

- The input contains a positive integer n.

Output Format:

- Contains the list of prime numbers up to n

Example:

Input:

10

Output:

[2, 3, 5, 7]

Constraints:

- The input n should be a positive integer greater than or equal to 2.
- The algorithm should be efficient, aiming for a time complexity of approximately $O(n * sqrt(n))$ where n is the input number.
- The algorithm should handle large inputs reasonably well, without consuming excessive memory or taking an unreasonable amount of time to execute.

Source Code:

CTP32252.py

```
a=int(input())
myList=[]
for i in range(2,a+1):
        count=0
        for j in range(1,i+1):
                if i%j==0:
                        count=count+1
        if count==2:
                myList.append(i)
print(myList)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
10
[2, 3, 5, 7]

Test Case - 2
User Output
20
[2, 3, 5, 7, 11, 13, 17, 19]

Test Case - 3
User Output
30
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

S.No: 3	Exp. Name: ***Swap two numbers***	Date: 2026-08-04

Aim:

Write a Python program that reads two numbers from the user and swaps their values using only two variables.

Source Code:

swapping.py

```
a=float(input("Enter the first number: "))
b=float(input("Enter the second number: "))
print("Original numbers:")
print("First number:",a)
print("Second number:",b)
a=a+b
b=a-b
a=a-b
print("Swapped numbers:")
print("First number:",a)
print("Second number:",b)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the first number:
2
Enter the second number:
3
Original numbers:
First number: 2.0
Second number: 3.0
Swapped numbers:
First number: 3.0
Second number: 2.0

Test Case - 2

User Output
Enter the first number:
-3.5
Enter the second number:
2.5
Original numbers:
First number: -3.5
Second number: 2.5
Swapped numbers:
First number: 2.5
Second number: -3.5

S.No: 4	Exp. Name: ***Arithmetic Operations (Addition, Subtraction, Multiplication, Division)***	**Date: 2026-08-04**

Aim:

Write a program to read two positive integer inputs from the user and perform the following arithmetic operations: addition, subtraction, multiplication, and division, and print the result.

Input Format:

- The first two lines represent the input of two integers $num1$ and $num2$.

Output Format:

- The first line should print the sum of the provided inputs.
- The second line should print the subtraction of the provided inputs.
- The third line should print the multiplication of the provided inputs.
- The fourth line should print the division of the provided inputs.

Sample Test Case:

Input:

```
num1: 40
num2: 20
```

Output:

```
Addition of 40 and 20 = 60
Subtraction of 40 and 20 = 20
Multiplication of 40 and 20 = 800
Division of 40 and 20 = 2.0
```

Note:

- Assume the value of $num2$ is always greater than 0 (zero).

Source Code:

Arithexample2.py

```
a=int(input("num1: "))
b=int(input("num2: "))
print("Addition of",a,"and",b,"=",a+b)
print("Subtraction of",a,"and",b,"=",a-b)
print("Multiplication of",a,"and",b,"=",a*b)
print("Division of",a,"and",b,"=",a/b)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
num1:
40
num2:
20
Addition of 40 and 20 = 60
Subtraction of 40 and 20 = 20
Multiplication of 40 and 20 = 800
Division of 40 and 20 = 2.0

Test Case - 2
User Output
num1:
4
num2:
6
Addition of 4 and 6 = 10
Subtraction of 4 and 6 = -2
Multiplication of 4 and 6 = 24
Division of 4 and 6 = 0.6666666666666666

S.No: 5	Exp. Name: ***Logical Operators***	Date: 2026-08-04

Aim:

Write a Python program that reads two integers and performs logical operations using the `and`, `or`, and `not` operators.

Input Format:

- The first line contains an integer $num1$.
- The second line contains an integer $num2$.

Output Format:

- Print the results of the logical operations in the following format:

```
<num1> and <num2>: <result>
<num1> or <num2>: <result>
not <num1>: <result>
not <num2>: <result>
```

Note: Refer to the visible test cases for better understanding.

Source Code:

logicoperator.py

```
a=int(input())
b=int(input())
print(a,"and",b,end='')
print(":",a and b)
print(a,"or",b,end='')
print(":",a or b)
print("not",a,end='')
print(":",not a)
print("not",b,end='')
print(":",not b)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
4

3 and 4: 4
3 or 4: 3
not 3: False
not 4: False

Test Case - 2
User Output
0
1
0 and 1: 0
0 or 1: 1
not 0: True
not 1: False

Test Case - 3
User Output
15
45
15 and 45: 45
15 or 45: 15
not 15: False
not 45: False

S.No: 6	Exp. Name: ***Comparison Operators***	Date: 2026-08-04

Aim:

Write a Python program that reads two integers and demonstrates the use of comparison operators by performing the six comparison operations (==, !=, <, <=, >, and >=).

Input Format:

- The first line contains an integer $num1$.
- The second line contains an integer $num2$.

Output Format:

- Print the results of all six comparison operations, each on a separate line in the following format:

```
<num1> == <num2> : <boolean result>
<num1> != <num2> : <boolean result>
<num1> < <num2> : <boolean result>
<num1> <= <num2> : <boolean result>
<num1> > <num2> : <boolean result>
<num1> >= <num2> : <boolean result>
```

Source Code:

operators.py

```
#write your code here
num1=int(input())
num2=int(input())
print(f"{num1} == {num2} :",num1==num2)
print(f"{num1} != {num2} :",num1!=num2)
print(f"{num1} < {num2}  :",num1<num2)
print(f"{num1} <= {num2} :",num1<=num2)
print(f"{num1} > {num2}  :",num1>num2)
print(f"{num1} >= {num2} :",num1>=num2)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
4
4 == 4 : True

4 != 4 : False
4 < 4 : False
4 <= 4 : True
4 > 4 : False
4 >= 4 : True

Test Case - 2
User Output
3
2
3 == 2 : False
3 != 2 : True
3 < 2 : False
3 <= 2 : False
3 > 2 : True
3 >= 2 : True

Test Case - 3
User Output
65
76
65 == 76 : False
65 != 76 : True
65 < 76 : True
65 <= 76 : True
65 > 76 : False
65 >= 76 : False

S.No: 7	Exp. Name: ***Writing an example using identity operator "is not"***	**Date: 2026-08-04**

Aim:

Take two integers `x` and `y` as input from the console using `input()` function. Using the identity operator `is not` check if `x` and `y` are same objects or not, print to the console, the result of the two input integers as shown in the example.

Sample Input and Output 1:

```
x: -8
y: -8
-8 is not -8 True
```

Sample Input and Output 2:

```
x: 256
y: 256
256 is not 256 False
```

Source Code:

IdenopExample3.py

```
# Take user input for x
x=int(input("x: "))
# Take user input for y
y=int(input("y: "))
# check whether x is not y and print the result
print(x, 'is not', y,x is not y)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
x:
-8
y:

-8
-8 is not -8 True

Test Case - 2
User Output
x:
256
y:
256
256 is not 256 False

S.No: 8	Exp. Name: ***Bitwise Operator***	Date: 2026-08-04

Aim:

Take two integers x and y as inputs from the console using input() function. For each bitwise operator (>> , <<, &, |, ~, and ^), print to the console, the result of applying these operators on the two input integers as shown in the example.

Sample Input and Output:

```
x: 52
y: 20
52 >> 20 is 0
52 << 20 is 54525952
52 & 20 is 20
52 | 20 is 52
~ 52 is -53
52 ^ 20 is 32
```

Source Code:

Bitopexample1.py

```
x=int(input("x: "))
y=int(input("y: "))
print(x, ">>" ,y, "is" ,x>>y)
print(x, "<<" ,y, "is" ,x<<y)
print(x, "&" ,y, "is" ,x&y)
print(x, "|" ,y, "is" ,x|y)
print("~" ,x, "is" ,~x)
print(x, "^" ,y, "is" ,x^y)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
x:
30
y:

2
30 >> 2 is 7
30 << 2 is 120
30 & 2 is 2
30 | 2 is 30
~ 30 is -31
30 ^ 2 is 28

Test Case - 2
User Output
x:
10
y:
20
10 >> 20 is 0
10 << 20 is 10485760
10 & 20 is 0
10 | 20 is 30
~ 10 is -11
10 ^ 20 is 30

S.No: 9	Exp. Name: ***Write a program on assignment operators =,+=,-= and *=***	**Date: 2026-08-04**

Aim:

Take two integers `x` and `y` as inputs from the console using `input()` function. For each assignment operator (`=` , `+=`, `-=`, and `*=`), print to the console, the result of applying these operators on the two input integers as shown in the example.

Assumption:

- y is a non-zero integer

Sample Input and Output:

```
x: 20
y: 30
x = y: 30
x += y: 50
x -= y: -10
x *= y: 600
```

Hint: Before every assignment operation, reset the values of x and y to the initial values.

Source Code:

Assignexample2.py

```
# Assignment Operators =, +=, -=, *=
x=int(input("x: "))
y=int(input("y: "))
print("x = y:",y)
print("x += y:",x+y)
print("x -= y:",x-y)
print("x *= y:",x*y)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
x: 20

y:
30
x = y: 30
x += y: 50
x -= y: -10
x *= y: 600

Test Case - 2
User Output
x:
6
y:
3
x = y: 3
x += y: 9
x -= y: 3
x *= y: 18

S.No: 10	Exp. Name: ***Ternary Operator***	Date: 2026-08-04

Aim:

Write a python program to find the maximum value of given two numbers using ternary operator.

Input Format:

- First line should prompt the user to enter first integer
- Second line should prompt the user to enter second integer

Output Format:

- Output should print the maximum value using ternary operator

Source Code:

ternary.py

```
x = int(input())
y = int(input())
max_value =x if(x>y) else y
print(max_value)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
91
123
123

Test Case - 2
User Output
15
-23
15

S.No: 11	Exp. Name: ***Membership Operator***	Date: 2026-08-04

Aim:

Write a Python program to demonstrate the use of the membership operators `in` and `not in` with a predefined list of integers.

Input Format:

- A single line contains an integer n to be checked for membership in the predefined list.

Output Format:

- Print "True" or "False" indicating whether n is present in the list using the in operator.
- Print "True" or "False" indicating whether n is not present in the list using the not in operator.

Note:

- The predefined list of integers is already provided in the code.
- Refer to the visible test cases for better understanding.

Source Code:

membership.py

```
num_list = [1, 2, 3, 4, 5, 15, 25, 89, 54, 123, 489]
a=int(input())
print(a in num_list)
print(a not in num_list)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
21
False
True

Test Case - 2
User Output
-3
False
True

Test Case - 3
User Output
5
True
False

Test Case - 4
User Output
54
True
False

S.No: 12	Exp. Name: ***Arithmetic Operations on Complex Numbers***	**Date: 2026-08-04**

Aim:

Python provides built-in support for complex numbers. Once a complex number is created, it behaves like any other numeric data type, allowing you to perform arithmetic operations using standard operators.

A complex number can be entered using the `complex()` function.

Syntax:

```
complex(input())
```

When the user enters a value such as 3+4j or 10-2j, the complex() function converts the input string into a complex number.

Example:

```
c = complex(input("Enter a complex number: "))
print(c)
print(type(c))
```

Sample Input:

```
5+2j
```

Output:

```
(5+2j)
<class 'complex'>
```

Python supports all common arithmetic operators on complex numbers.

Example:

```
c1 = 2 + 3j
c2 = 1 + 4j
print(c1 + c2)
print(c1 - c2)
print(c1 * c2)
print(c1 / c2)
```

Python automatically computes the correct complex result for each operation.

Task

Take two complex numbers as input from the console. For each arithmetic operator(+, -, *, and /), perform these operations on the two complex numbers and print the result as shown in the example.

Sample Input and Output:

```
a1: 10+3j
b2: 14+3j
a1 + b2 = (24+6j)
a1 - b2 = (-4+0j)
a1 * b2 = (131+72j)
a1 / b2 = (0.7268292682926829+0.05853658536585366j)
```

Source Code:

ComplexNums.py

```
# Take input for the first complex number a1
a1=complex(input("a1: "))
# Take input for the second complex number b2
b2=complex(input("b2: "))

# perform + operation on the complex numbers and print the result
print("a1 + b2 =",a1+b2)

# subtract b2 from a1 and print the result
print("a1 - b2 =",a1-b2)

# multiple a1, b2 and print the result
print("a1 * b2 =",a1*b2)

# divide b2 from a1 and print the result
print("a1 / b2 =",a1/b2)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
a1:
10+3j
b2:
41+3j
a1 + b2 = (51+6j)
a1 - b2 = (-31+0j)
a1 * b2 = (401+153j)
a1 / b2 = (0.24792899408284022+0.05502958579881657j)

Test Case - 2
User Output
a1:
45+9j
b2:
15+8j
a1 + b2 = (60+17j)
a1 - b2 = (30+1j)
a1 * b2 = (603+495j)
a1 / b2 = (2.58477508650519-0.7785467128027682j)

S.No: 13	Exp. Name: ***Print Multiplication Table***	Date: 2026-08-04

Aim:

Write a Python program to print the multiplication table of a given number in the given format.

Input Format:

- Read an integer n with a prompt "Enter a number : ".

Output Format:

- Print the multiplication table of n from 1 to 10.
- Each line should contain the value of n, the multiplier i, and the product in the following format:

`<n>*<i>=<result>`

where,i ranges from 1 to 10, and $result$ represents $n \times i$.

Note: Refer to visible test cases for better understanding of input and output formats.

Source Code:

multiplication_table.py

```
n=int(input("Enter a number : "))
for i in range(1,11):
        print(n,end='')
        print("*",end='')
        print(i,end='')
        print("=",end='')
        print(n*i)
        i=i+1
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a number :
2
2*1=2
2*2=4
2*3=6

2*4=8
2*5=10
2*6=12
2*7=14
2*8=16
2*9=18
2*10=20

Test Case - 2
User Output
Enter a number :
13
13*1=13
13*2=26
13*3=39
13*4=52
13*5=65
13*6=78
13*7=91
13*8=104
13*9=117
13*10=130

Test Case - 3
User Output
Enter a number :
-10
-10*1=-10
-10*2=-20
-10*3=-30
-10*4=-40
-10*5=-50
-10*6=-60
-10*7=-70
-10*8=-80
-10*9=-90
-10*10=-100